

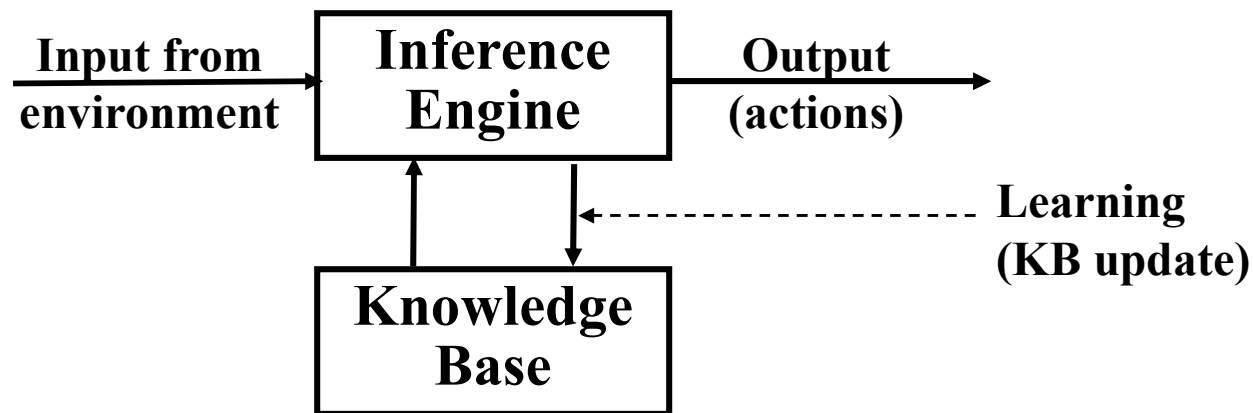
Logical Agents

Chapter 7

A Knowledge-Based Agent

- A knowledge-based agent consists of a knowledge base (KB) and an inference engine (IE).
- A knowledge-base is a set of representations of what one knows about the world (objects and classes of objects, the fact about objects, relationships among objects, etc.)
- Each individual representation is called a sentence.
- The sentences are expressed in a knowledge representation language.
- Examples of sentences
 - The moon is made of green cheese
 - If A is true then B is true
 - A is false
 - All humans are mortal
 - Confucius is a human

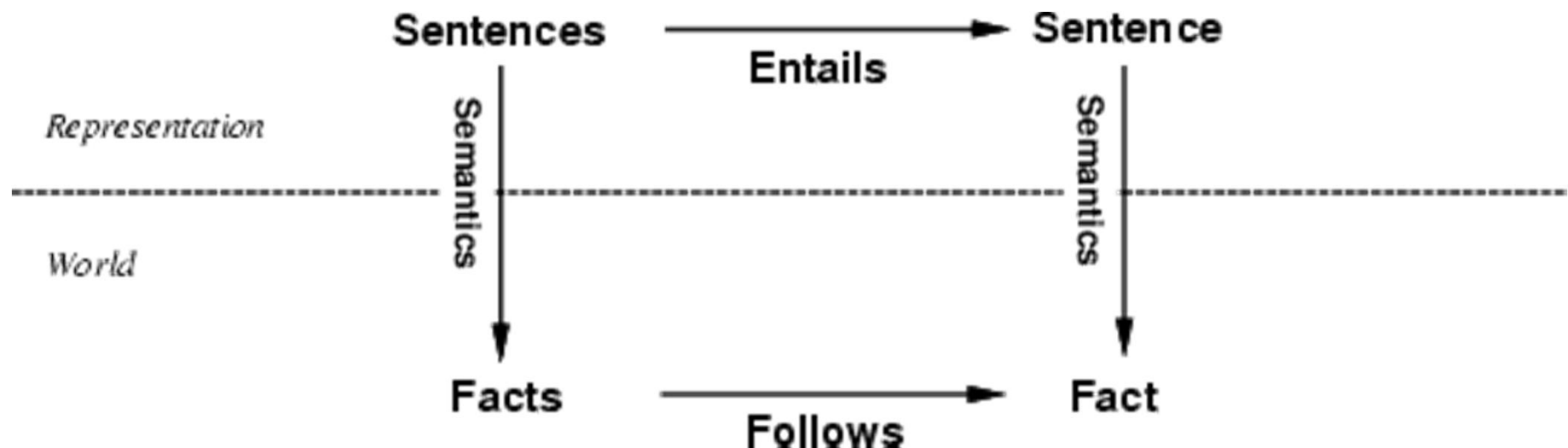
- The Inference engine derives new sentences from the input and KB
- The inference mechanism depends on representation in KB
- The agent operates as follows:
 1. It receives percepts from environment
 2. It computes what action it should perform (by IE and KB)
 3. It performs the chosen action (some actions are simply inserting inferred new facts into KB).



Representation, Reasoning, and Logic

- The objective of knowledge representation is to express knowledge in a computer-tractable form, so that agents can perform well.
- A knowledge representation language is defined by:
 - Its **syntax** which defines all possible sequences of symbols that constitute sentences of the language (**grammar** to form sentences)
 - Its **semantics** determines the facts in the world to which the sentences refer (**meaning** of sentences)
 - Each sentence makes a claim about the world.
 - Its **proof theory** (inference rules and proof procedures)

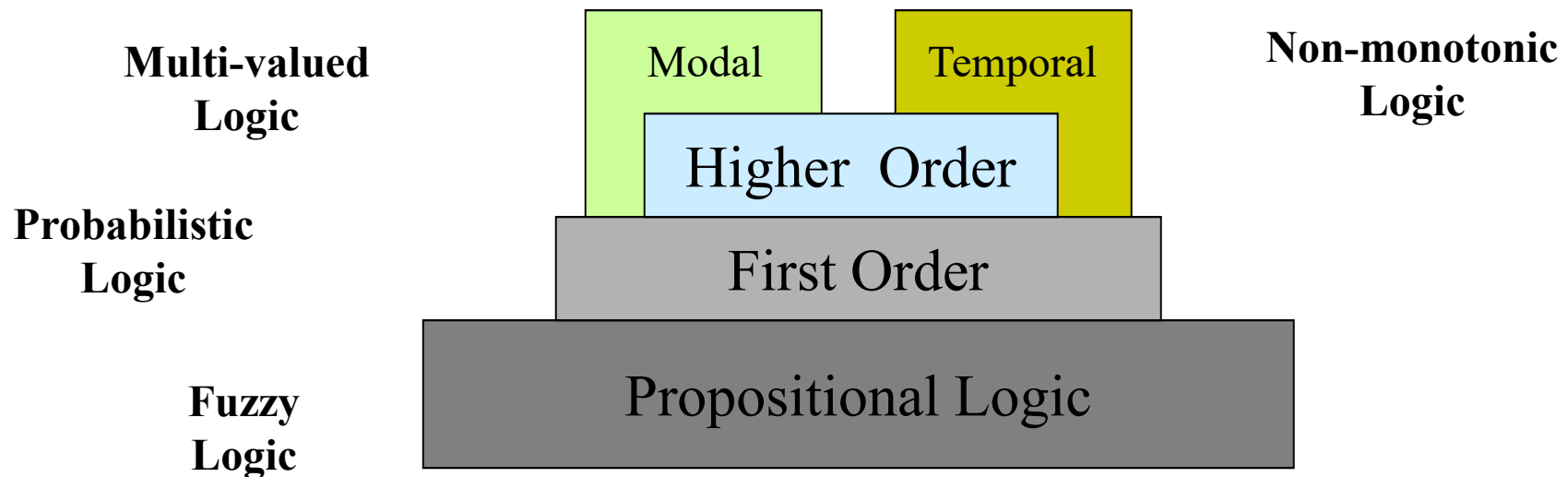
The Connection between Sentences and Facts



Semantics maps sentences in logic to facts in the world.

The property of one fact following from another is mirrored by the property of one sentence being entailed by (inferred from) another.

Logic as a KR language



Propositional Logic: Syntax

- **Symbols:**

Logical constants: true (T), false (F)

Propositional symbols: P, Q, S, ...

logical connectives:

$\wedge$	...conjunction (and)	$\vee$	...disjunction (or)
$\sim$	...negation (not)	$\Rightarrow$	...implication (if)
$\Leftrightarrow$	...logical equivalence (if and only if)		

Wrapping **parentheses**: (...)

- A **proposition** (denoted by a proposition symbol) is a declarative statement which can be either true or false but not both or neither.
 - The moon is made of green cheese (F)
 - UMBC is closer to Baltimore than to Washington, DC (T)
 - $P = NP$ (truth unknown)

- **Sentence**

1. T or F itself is a sentence
2. Individual proposition symbols P, Q, ... are sentences
3. If S is a sentence, so is (S)
4. If S1 and S2 are sentences, so are
 $S1 \wedge S2, S1 \vee S2, S1 \Rightarrow S2, S1 \Leftrightarrow S2, \sim S1$
5. Nothing else is a sentence

- **Order of precedence** of logical connectors

$\sim, \wedge, \vee, \Rightarrow, \Leftrightarrow$

- Minimum set of logical connectors

$(\sim, \wedge)$, or $(\sim, \vee)$

- **Atomic sentences**: T, F, P, Q, ...

- **Literals**: atomic sentences and their negations

Propositional logic: **Syntax**

Sentence $\rightarrow$ *AtomicSentence* | *ComplexSentence*

AtomicSentence $\rightarrow$ *True* | *False* | *P* | *Q* | *R* | ...

ComplexSentence $\rightarrow$ (*Sentence*) | [*Sentence*]

| $\neg$ *Sentence*

| *Sentence* $\wedge$ *Sentence*

| *Sentence* $\vee$ *Sentence*

| *Sentence* $\Rightarrow$ *Sentence*

| *Sentence* $\Leftrightarrow$ *Sentence*

OPERATOR PRECEDENCE : $\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow$

Figure 7.7 A BNF (Backus–Naur Form) grammar of sentences in propositional logic, along with operator precedences, from highest to lowest.

Examples of PL sentences

- P means "It is hot"
- Q means "It is humid"
- R means "It is raining"

- $P \wedge Q \Rightarrow R$
"If it is hot and humid, then it is raining"
- $Q \Rightarrow P$
"If it is humid, then it is hot"
- Q
"It is humid."

Propositional Logic (PL): Semantics

- Need an **interpretation** of symbols for a given set of sentences
 - Proposition symbols do not have meaning by themselves
 - An interpretation connects proposition symbols to a statement about the world (which may be true or false in that world)
 - An **interpretation** in PL can be defined as an **assignment of truth values** to all proposition symbols involved
 - There are many interpretations for a given set of sentences (2^n if they involve n distinct proposition symbols)
 - Example:
 - I_1: P: it is humid (T) Q: it is hot (T) .
 $P \wedge Q$ is true
 - I_2: P: moon is made of green cheese (F); Q: I am happy (T).
 $P \wedge Q$ is false

- Give a meaning to each logical connectives
 - A connective is a function (boolean),
 - It does not depend on any particular interpretations (universal to all PL sentences)
 - It can best be defined by a truth table
- Truth value of each sentence can then be calculated

Truth tables for logical connectives

P	Q	$\sim P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$
T	T	F	T	T	T	T
T	F	F	F	T	F	F
F	T	T	F	T	T	F
F	F	T	F	F	T	T

- Equivalence laws in PL (both sides have the same truth table)

- $P \Rightarrow Q \equiv \sim P \vee Q$

- $P \Leftrightarrow Q \equiv (P \Rightarrow Q) \wedge (Q \Rightarrow P)$

- Distribution law

- $P \wedge (Q \vee R) \equiv (P \wedge Q) \vee (P \wedge R)$

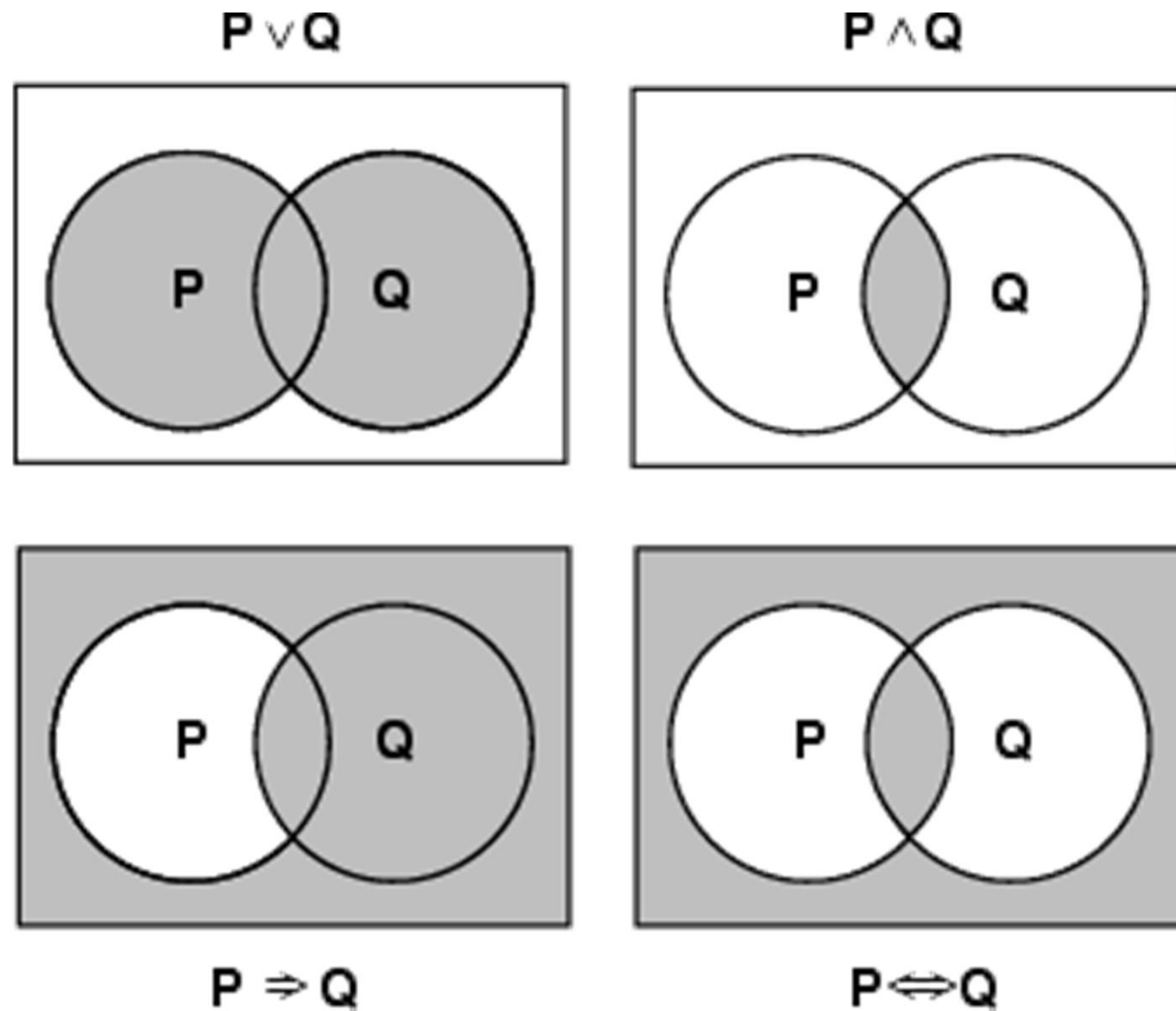
- $P \vee (Q \wedge R) \equiv (P \vee Q) \wedge (P \vee R)$

- De Morgan's law

- $\sim(P \vee Q \vee R) \equiv \sim P \wedge \sim Q \wedge \sim R$

- $\sim(P \wedge Q \wedge R) \equiv \sim P \vee \sim Q \vee \sim R$

Models of Complex Sentences (Venn diagrams)



Fundamental concepts in logic and proof theory

- **Entailment ($\models$):** Entailment is a semantic relation between sentences. We say that a set of sentences KB entails a sentence Q (written as $\text{KB} \models Q$) if Q is true in every model in which all the sentences in KB are true.
- Example: $\{P, P \rightarrow Q\} \models Q$.
- This means that in any interpretation where P is true and $P \rightarrow Q$ is true, Q must also be true.

- **Soundness:** A logical system is sound if every provable statement is true in all models.
- In other words, if we can derive Q from KB using the rules of our system (written as $KB \vdash Q$), then KB must entail Q ($KB \models Q$).
- Formally: If $KB \vdash Q$, then $KB \models Q$.
- Soundness ensures that our proof system doesn't allow us to prove things that aren't actually logically entailed.

Completeness:

- A logical system is complete if every valid statement is provable within the system. In other words, if KB entails Q, then we should be able to derive Q from KB using the rules of our system.
- Formally: If $KB \models Q$, then $KB \vdash Q$
- Completeness ensures that our proof system is powerful enough to prove everything that is logically entailed.

Validity:

- A formula is valid if it is true under all possible interpretations. Valid formulas are also called tautologies.
- Example: $P \vee \neg P$ is valid because it's true regardless of whether P is true or false.

Satisfiability:

- A formula is satisfiable if there exists at least one interpretation (model/world) that makes it true.
- Example: $P \wedge Q$ is satisfiable because it's true when both P and Q are true.

Consistency:

- A set of formulas is consistent if there exists an interpretation that makes all the formulas in the set true simultaneously. If no such interpretation exists, the set is inconsistent.
- Example: $\{P, P \rightarrow Q, \neg Q\}$ is inconsistent because there's no way to assign truth values to P and Q that makes all three statements true.

Inference by enumeration

- We want to see if α is entailed by KB
- Enumeration of all models is sound and complete.
- But...for n symbols, time complexity is $O(2^n)$...
- We need a more efficient way to do inference
 - But worst-case complexity will remain exponential for propositional logic

Inference Rule

- **Logical Inference** is used to create new sentences X that *logically follow* from a given set of sentences S (in KB and from input), i.e., $S \models X$
- This kind of inference is also known as **deduction**
- **Use truth table:** Whether $S \models X$ holds can be determined by whether $S \Rightarrow X$ is a tautology
 - $S \models X$ holds iff every interpretation I that makes S true also makes X true
 - $S \Rightarrow X$ is a tautology iff every interpretation I that makes S true also makes X true
 - Example: $P, P \Rightarrow Q \models Q$ because $(P, P \Rightarrow Q) \Rightarrow Q$ is true under any interpretation
 - Huge truth tables for inference involving large number of sentences

- **Use inference rules:** generate new sentences X from a one or more existing sentences S . S is called the premise and X the conclusion of the rule.
- **Proof procedure:** a set of inference rules and a procedure of how to use these rules
- If X can be generated from S by proof procedure i , we say X is derived from S by i , denoted $S \mid_i X$, or $S \vdash X$.
- **Soundness.** An inference procedure is sound if every sentence X it produces from a set of sentences S logically follows from S . (No contradiction is created).

For all S and X , if $S \vdash X$ then $S \models X$

- **Completeness.** A inference procedure is complete, if it is able to produce every sentence that logically follows from any give S .

For all S and X , if $S \models X$ then $S \vdash X$

1. Soundness:

- A formal system is **sound** if every inference it makes is logically valid. In other words, if the system derives a conclusion from a set of premises, then that conclusion must be true whenever the premises are true.
- Mathematically, for a sound system, if $\Gamma \vdash \varphi$ (i.e., φ is derivable from Γ using the inference rules), then $\Gamma \models \varphi$ (i.e., φ is semantically entailed by Γ).

Soundness ensures that the system does not derive any false conclusions.

Everything provable in the system is guaranteed to be true in the logical sense.

2. Completeness:

- A formal system is **complete** if every logically valid statement can be derived using the system's inference rules. That is, if a conclusion is true in every model where the premises are true, the system should be able to derive it.
- Mathematically, for a complete system, if $\Gamma \models \varphi$ (i.e., φ is semantically entailed by Γ), then $\Gamma \vdash \varphi$ (i.e., φ is derivable from Γ using the inference rules).

Completeness ensures that the system can prove everything that is true in the logic.

Summary:

Soundness: Everything that is provable is true.

Completeness: Everything that is true is provable.

A system with both soundness and completeness provides a robust and reliable framework for inference, meaning it neither proves false statements (soundness) nor fails to prove true ones (completeness).

Sound Inference Rules (deductive rules)

- Here are some examples of **sound** rules of inference.
- Each can be shown to be sound using a truth table -- a rule is sound if it's conclusion is true whenever the premise is true.

<u>RULE</u>	<u>PREMISES</u>	<u>CONCLUSION</u>
Modus Ponens	$A, A \Rightarrow B$	B
Modus Tollens	$\sim B, A \Rightarrow B$	$\sim A$
And Introduction	A, B	$A \wedge B$
And Elimination	$A \wedge B$	A
Or Introduction	A	$A \vee B$
Double Negation	$\sim\sim A$	A
Chaining	$A \Rightarrow B, B \Rightarrow C$	$A \Rightarrow C$

• Resolution rule

Unit Resolution $A \vee B, \sim A \vdash B$

Resolution $A \vee B, \sim B \vee C \vdash A \vee C$

Let $\alpha_1 \dots \alpha_i, \beta, \gamma_1 \dots \gamma_k$ be literals. Then

$$\alpha_1 \vee \dots \vee \alpha_i \vee \beta, \sim \beta \vee \gamma_1 \vee \dots \vee \gamma_k \vdash \alpha_1 \vee \dots \vee \alpha_i \vee \gamma_1 \vee \dots \vee \gamma_k$$

- Operates on two disjunctions of literals
- The pair of two opposite literals (β and $\sim \beta$) cancel each other, all other literals from the two disjunctions are combined to form a new disjunction as the inferred sentence
- Resolution rule can replace all other inference rules

Modus Ponens $A, \sim A \vee B \vdash B$

Modus Tollens $\sim B, \sim A \vee B \vdash \sim A$

Chaining $\sim A \vee B, \sim B \vee C \vdash \sim A \vee C$

Soundness of the Resolution Inference Rule

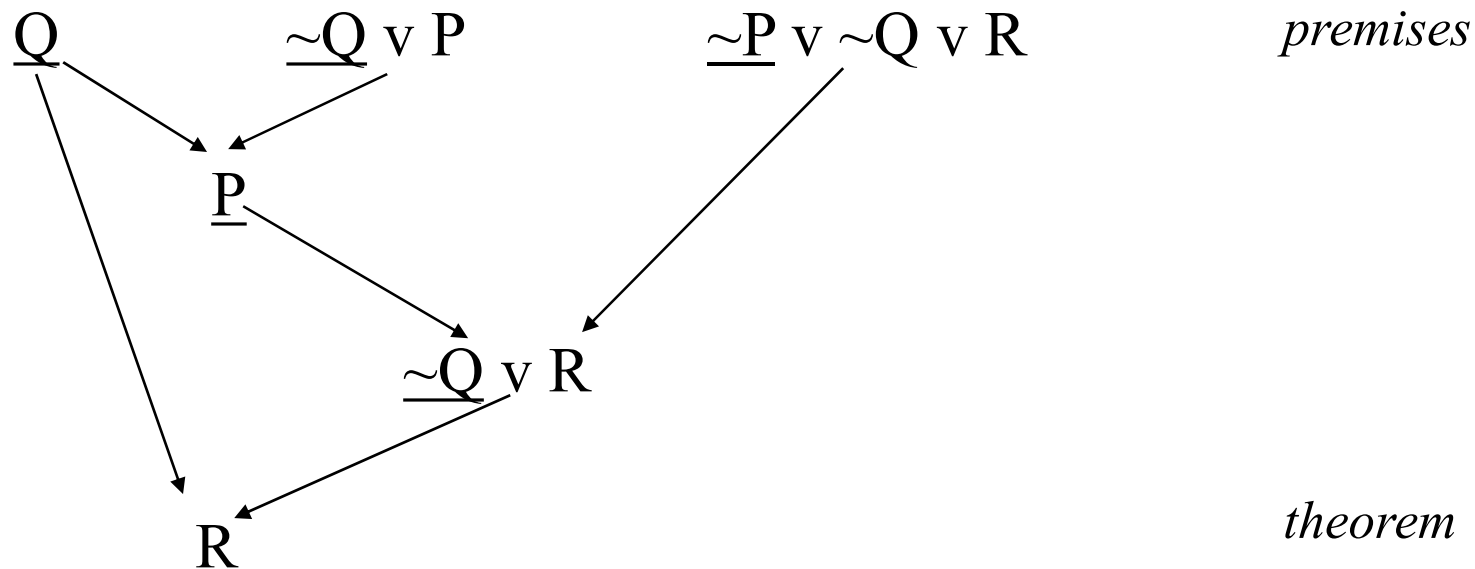
α	β	γ	$\alpha \vee \beta$	$\neg\beta \vee \gamma$	$\alpha \vee \gamma$
<i>False</i>	<i>False</i>	<i>False</i>	<i>False</i>	<i>True</i>	<i>False</i>
<i>False</i>	<i>False</i>	<i>True</i>	<i>False</i>	<i>True</i>	<i>True</i>
<i>False</i>	<i>True</i>	<i>False</i>	<i>True</i>	<i>False</i>	<i>False</i>
<u><i>False</i></u>	<u><i>True</i></u>	<u><i>True</i></u>	<u><i>True</i></u>	<u><i>True</i></u>	<u><i>True</i></u>
<u><i>True</i></u>	<u><i>False</i></u>	<u><i>False</i></u>	<u><i>True</i></u>	<u><i>True</i></u>	<u><i>True</i></u>
<u><i>True</i></u>	<u><i>False</i></u>	<u><i>True</i></u>	<u><i>True</i></u>	<u><i>True</i></u>	<u><i>True</i></u>
<i>True</i>	<i>True</i>	<i>False</i>	<i>True</i>	<i>False</i>	<i>True</i>
<u><i>True</i></u>	<u><i>True</i></u>	<u><i>True</i></u>	<u><i>True</i></u>	<u><i>True</i></u>	<u><i>True</i></u>

Proving things

- A **proof** is a sequence of sentences, where each sentence is either a premise (also called an axiom) or a sentence derived from earlier sentences in the proof by one of the rules of inference.
- The last sentence is the **theorem** that we want to prove (also called goal or query) .
- Complete proof procedure for PL has time complexity exponential to the number of premises
- Example for the "weather problem" given before.

1 Q	Premise	"It is humid"
2 $Q \Rightarrow P$	Premise	"if it is humid, it is hot"
3 P	Modus Ponens(1,2)	"It is hot"
4 $(P \wedge Q) \Rightarrow R$	Premise	"If it's hot & humid, it's raining"
5 $P \wedge Q$	And Introduction(1, 3)	"It is hot and humid"
6 R	Modus Ponens(4,5)	"It is raining"

- Proof by resolution



- Theorem proving as search

- Start node: the set of given premises/axioms (KB + Input)
- Operator: inference rule (add a new sentence into parent node)
- Goal: a state that contains the theorem asked to prove
- Solution: a path from start node to a goal

Normal forms of PL sentences

- Disjunctive normal form (DNF)
 - Any sentence can be written as a disjunction of conjunctions of literals.
 - Examples: $\underline{P \wedge Q \wedge \sim R}$; $\underline{A \wedge B} \vee \underline{C \wedge D} \vee \underline{P \wedge Q \wedge R}$; $\underline{P}$
 - Widely used in logical circuit design (simplification)
- Conjunctive normal form (CNF)
 - Any sentence can be written as a conjunction of disjunctions of literals.
 - Examples: $\underline{P \vee Q \vee \sim R}$; $(\underline{A \vee B}) \wedge (\underline{C \vee D}) \wedge (\underline{P \vee Q \vee R})$; $\underline{P}$
- Normal forms can be obtained by applying equivalence laws

$$\begin{aligned} & [(A \vee B) \Rightarrow (C \vee D)] \Rightarrow P \\ \equiv & \sim[\sim(A \vee B) \vee (C \vee D)] \vee P \\ \equiv & [\sim\sim(A \vee B) \wedge \sim(C \vee D)] \vee P \\ \equiv & [(A \vee B) \wedge (\sim C \wedge \sim D)] \vee P \\ \equiv & (A \vee B \vee P) \wedge (\sim C \wedge \sim D \vee P) \\ \equiv & (A \vee B \vee P) \wedge (\sim C \vee P) \wedge (\sim D \vee P) \quad \text{a CNF} \end{aligned}$$

Normal Clausal Form

Eventually we
want to prove:

Knowledge base KB entails sentence α

We first rewrite

into **conjunctive normal form (CNF)**.

A “conjunction of disjunctions”

$$(A \vee \neg B) \wedge (B \vee \neg C \vee \neg D)$$

Clause

Clause

literals

- **Theorem: Any KB can be converted into an equivalent CNF.**
- k-CNF: exactly k literals per clause

Example: Conversion to CNF

$$B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$$

1. Eliminate $\Leftrightarrow$, replacing $\alpha \Leftrightarrow \beta$ with $(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)$.
 $(B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1})$
2. Eliminate $\Rightarrow$, replacing $\alpha \Rightarrow \beta$ with $\neg\alpha \vee \beta$.
 $(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg(P_{1,2} \vee P_{2,1}) \vee B_{1,1})$
3. Move $\neg$ inwards using de Morgan's rules and double-negation:
 $(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge ((\neg P_{1,2} \wedge \neg P_{2,1}) \vee B_{1,1})$
4. Apply distributive law ($\wedge$ over $\vee$) and flatten:
 $(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg P_{1,2} \vee B_{1,1}) \wedge (\neg P_{2,1} \vee B_{1,1})$

Logical equivalence

- Two sentences are **logically equivalent** iff they are true in same models: $\alpha \equiv \beta$ iff $\alpha \models \beta$ and $\beta \models \alpha$

$$(\alpha \wedge \beta) \equiv (\beta \wedge \alpha) \quad \text{commutativity of } \wedge$$

$$(\alpha \vee \beta) \equiv (\beta \vee \alpha) \quad \text{commutativity of } \vee$$

$$((\alpha \wedge \beta) \wedge \gamma) \equiv (\alpha \wedge (\beta \wedge \gamma)) \quad \text{associativity of } \wedge$$

$$((\alpha \vee \beta) \vee \gamma) \equiv (\alpha \vee (\beta \vee \gamma)) \quad \text{associativity of } \vee$$

$$\neg(\neg\alpha) \equiv \alpha \quad \text{double-negation elimination}$$

$$(\alpha \Rightarrow \beta) \equiv (\neg\beta \Rightarrow \neg\alpha) \quad \text{contraposition}$$

$$(\alpha \Rightarrow \beta) \equiv (\neg\alpha \vee \beta) \quad \text{implication elimination}$$

$$(\alpha \Leftrightarrow \beta) \equiv ((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)) \quad \text{biconditional elimination}$$

$$\neg(\alpha \wedge \beta) \equiv (\neg\alpha \vee \neg\beta) \quad \text{de Morgan}$$

$$\neg(\alpha \vee \beta) \equiv (\neg\alpha \wedge \neg\beta) \quad \text{de Morgan}$$

$$(\alpha \wedge (\beta \vee \gamma)) \equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma)) \quad \text{distributivity of } \wedge \text{ over } \vee$$

$$(\alpha \vee (\beta \wedge \gamma)) \equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma)) \quad \text{distributivity of } \vee \text{ over } \wedge$$

Forward reasoning and backward reasoning

- Forward reasoning and backward reasoning (also called forward and backward chaining) are **two common approaches used in propositional logic for deriving conclusions from a set of premises.**
- They represent two different ways of applying inference rules to **either deduce new facts (forward) or verify a hypothesis (backward).**

Forward Reasoning (Forward Chaining)

- In forward reasoning, you start with known facts or premises and apply inference rules to derive new conclusions. You continue to apply the rules until you either derive the desired conclusion or exhaust all possibilities.
- It is **data-driven** and is commonly used when you have a set of premises and want to explore what can be inferred from them.
- **Steps:**
 - Start from the initial facts.
 - Apply inference rules to these facts to infer new facts.
 - Repeat the process until the goal is reached or no more facts can be inferred.

Example 1: Consider the following premises:

- $A \rightarrow B$ (If A is true, then B is true.)
- $B \rightarrow C$ (If B is true, then C is true.)
- A is true.
- Using forward reasoning:
 - Since A is true, by $A \rightarrow B$, we can infer that B is true.
 - Now that B is true, by $B \rightarrow C$, we can infer that C is true.
- We started with A and inferred B, then C using forward chaining.

- Example 2 (Practical Example - Medical Diagnosis):
 - Cough→Cold
 - Cold→Fever
 - Fever→Flu
 - Patient has a cough.
- Using forward reasoning:
 - The patient has a cough, so we infer they might have a cold.
 - If the patient has a cold, they might develop a fever.
 - If the patient has a fever, they might have the flu.

Backward Reasoning (Backward Chaining)

- In backward reasoning, you start with a desired conclusion or goal (hypothesis) and work backward to see if it can be derived from known premises.
- It is **goal-driven** and is useful when you have a specific conclusion in mind and want to verify whether it can be derived from the given facts.

Steps:

- Start from the goal (conclusion) you want to prove.
- Look for premises that can lead to this goal.
- Verify whether the premises are true, and repeat the process until you reach known facts or disprove the goal.

- **Example 1:** Consider the same premises:

- $A \rightarrow B$

- $B \rightarrow C$

- A is true.

- Goal: Prove C.

Using backward reasoning:

- To prove C, we check whether $B \rightarrow C$. So, we need to prove B.

- To prove B, we check $A \rightarrow B$. So, we need to prove A.

- Since A is given as true, we can now prove that B is true.

- Since B is true, we can then conclude that C is true.

Horn Clause

- A Horn clause is a special type of clause in propositional or predicate logic that **contains at most one positive literal** (a literal is a variable or its negation).
- It is named after Alfred Horn, who first identified them.
- A clause is a disjunction of literals, and a Horn clause has the following form:
 - Definite Horn Clause: $L_1 \wedge L_2 \wedge \dots \wedge L_n \rightarrow L$
This can be rewritten as: $\neg L_1 \vee \neg L_2 \vee \dots \vee \neg L_n \vee L$
It has one positive literal L and several negative literals $\neg L_1, \neg L_2, \dots, \neg L_n$.
- Fact: A special case where the Horn clause has no negative literals, i.e., L , which is just a single positive literal.

Horn Sentence

- A Horn sentence is a conjunction of Horn clauses. It is a set of Horn clauses combined using the logical AND operator ($\wedge$).
- In other words, a Horn sentence is a logical formula where all the clauses are Horn clauses.

Usage of Horn Clauses and Horn Sentences:

In Propositional and Predicate Logic, Horn clauses are often used in automated theorem proving and model checking. They are computationally efficient for deduction because inference in Horn clauses can be performed in polynomial time.

Summary:

- **Horn Clause:** A disjunction of literals with at most one positive literal.
- **Horn Sentence:** A conjunction of Horn clauses.
- **Usage:** They are widely used in logic programming, automated reasoning, SAT problems, and AI systems for efficient logical deduction.

PL is Too Weak a Representational Language

- Consider the problem of representing the following information:
 - Every person is mortal. (S1)
 - Confucius is a person. (S2)
 - Confucius is mortal. (S3)
- S3 is clearly a logical consequence of S1 and S2. But how can these sentences be represented using PL so that we can infer the third sentence from the first two?
- We can use symbols P, Q, and R to denote the three propositions, but this leads us to nowhere because knowledge important to infer R from P and Q (i.e., relationship between being a human and mortality, and the membership relation between Confucius and human class) is not expressed in a way that can be used by inference rules

Weakness of PL

- Alternatively, we can use symbols for parts of each sentence
 - $P = \text{"person"}; M = \text{"mortal"}; C = \text{"Confucius"}$
 - The above 3 sentences can be roughly represented as:
 $S2: C \Rightarrow P; S1: P \Rightarrow M; S3: C \Rightarrow M.$
 - Then $S3$ is entailed by $S1$ and $S2$ by the chaining rule.
- Bad semantics
 - “Confucius” (and “person” and “mortal”) are not PL sentences (**not a declarative statement**) and cannot have a truth value.
 - What does $P \Rightarrow M$ mean?
- We need infinite distinct symbols X for individual persons, and infinite implications to connect these X with P (person) and M (mortal) because we need a unique symbol for each individual.
 $\text{Person_1} \Rightarrow P; \text{person_1} \Rightarrow M;$
 $\text{Person_2} \Rightarrow P; \text{person_2} \Rightarrow M;$
...
 $\text{Person_n} \Rightarrow P; \text{person_n} \Rightarrow M$

Weakness of PL

- Hard to identify "**individuals**." E.g., Mary, 3
 - Individuals cannot be PL sentences themselves.
- Difficult to directly and clearly talk about **properties** of individuals or **relations** between individuals (hard to connect individuals to class properties).
 - E.g., property of being a human implies property of being mortal
- Generalizations, patterns, regularities can't easily be represented.
 - All members of a class have this property
 - Some member of a class have this property
- A better representation is needed to capture the relationship (and distinction) between objects and classes, including properties belonging to classes and individuals
- First-Order Logic (abbreviated FOL or FOPC) is expressive enough to concisely represent this kind of situation by separating classes and individuals
 - Explicit representation of individuals and classes, x, Mary, 3, persons.
 - Adds relations, variables, and quantifiers, e.g.,
 - "*Every person is mortal*" Forall X: person(X) $\Rightarrow$ mortal(X)
 - "*There is a white alligator*" There exists some X: alligator(X) $\wedge$ white(X)